



COMSATS University Islamabad, Lahore Campus

CSC262 | Artificial Intelligence

exRES Project Report

Group Members

Name	Registration No
Ahmad Ali	FA24-BSE-012
Abdul Kareem	FA24-BSE-123
Muneeb Jillani Ijaz	FA24-BSE-147
Muhammad Haris	FA24-BSE-177

Course Instructor: Mr. Jawad Hassan
Date: May 21, 2026

Contents

1. Introduction	1
2. Dataset	1
2.1. Citation	1
3. Data Preprocessing	1
3.1. Shape of the Dataset	1
3.2. Filtering the Data	2
3.3. Validation Set	2
3.4. Computing Derived Features	3
3.5. Final Dataset	3
4. Model	4
4.1. Target Encoding	4
4.2. Feature Sets	4
4.3. Preprocessing Pipeline	4
4.3.1. Categorical Features	5
4.3.2. Numeric Features	5
4.4. Logistic Regression	5
4.5. Random Forest	6
4.6. XGBoost	6

1. Introduction

The English Premier League is one of the most popular football leagues in the world, and predicting the standings of the season is a challenging task. The project aims to use machine learning techniques to analyze historical data and make predictions about the individual match outcomes across a season and derive the final standings of the league by aggregating predicted results. The project involves data preprocessing, feature engineering, model selection, and evaluation. The goal is to develop a model that can accurately predict the final standings of the English Premier League based on historical match data.

exRES is a machine learning project to predict the standings of the 2024/2025 season of the English Premier League using historical data. exRES is short for Expected Result and takes inspiration from other football terms like Expected Goals (xG) and Expected Assists (xA).

2. Dataset

This project uses the [Club Football Match Data 2000-2025](#) dataset available on GitHub. The dataset contains match data from 25 years from many football leagues, including the English Premier League. The dataset is in the form of a csv file with over 230K records making it one of the largest free football datasets available. It contains full match data starting from 2000/01 and till the latest complete season (2024/25).

2.1. Citation

Author: Adam Gábor

ORCID: <https://orcid.org/0009-0001-9252-5976>

Affiliation: Faculty of Informatics and Information Technologies, Slovak University of Technology in Bratislava

Year: 2025

3. Data Preprocessing

The dataset was preprocessed to extract the required data for the English Premier League, filtering missing values and normalizing numerical features. The latest, complete season (2024/25) in the dataset was used as the validation set for the model while the rest was used for training.

3.1. Shape of the Dataset

The original dataset contained 230,557 rows and 48 columns. Here's a list of the columns in the dataset categorized into different types:

Category	Columns	
Match Metadata	Division	HomeTeam
	MatchTime	AwayTeam

Category	Columns	
	MatchDate	
Predictive Features	HomeElo	AwayElo
	Form3Home	Form5Home
	Form3Away	Form5Away
Match Result	FTHome	HTHome
	FTAway	HTAway
	FTResult	HTResult
Match Statistics	HomeShots	AwayShots
	HomeTarget	AwayTarget
	HomeFouls	AwayFouls
	HomeCorners	AwayCorners
	HomeYellow	AwayYellow
	HomeRed	AwayRed
Betting Odds	OddHome	OddDraw
	OddAway	MaxHome
	MaxDraw	MaxAway
	Over25	Under25
	MaxOver25	MaxUnder25
	HandiSize	HandiHome
	HandiAway	C_LTH
	C_LTA	C_VHD
	C_VAD	C-HTB
	C_PHB	

3.2. Filtering the Data

The dataset contained data from many football leagues all around the world. The first step was to filter the data to include only matches from the English Premier League. This was done by selecting records matching the Division column with the value “E0”.

```
ep1 = matches[matches["Division"] == "E0"].copy()
```

The first two seasons in the dataset (2000/01 and 2001/02) had missing values betting odds making it impossible to compute normalized implied probability features which are important for the model, so they were removed from the dataset.

Many columns were dropped from the dataset as they were not relevant for the model. These include match metadata, match statistics, and some betting odds.

3.3. Validation Set

The latest complete season in the dataset (2024/25) was used as the validation set for the model. It was separated from the training data to evaluate the model's performance on unseen data. It

has 380 matches which is the standard number of matches in a Premier League season (20 teams playing 38 matches each). No data from this season was used during training to ensure a fair evaluation of the model.

3.4. Computing Derived Features

Some features were derived from the existing data to provide more information to the model. These include:

- **EloDifference**: Computed from the difference of HomeElo and AwayElo.
- **Form3Difference**: Computed from the difference of Form3Home and Form3Away.
- **Form5Difference**: Computed from the difference of Form5Home and Form5Away.
- **NormProbHome, NormProbDraw, NormProbAway**: Normalized probabilities computed from the betting odds as the raw implied probabilities from the odds do not sum to 1 due to the bookmaker's margin. Normalization was done by dividing each implied probability by the sum of all three implied probabilities.

3.5. Final Dataset

After preprocessing, these two datasets were generated:

- **Training Set**: (8270, 16)
- **Validation Set**: (380, 16)

The following columns are in the final dataset:

1. MatchDate
2. HomeTeam
3. AwayTeam
4. HomeElo
5. AwayElo
6. EloDifference
7. Form3Home
8. Form3Away
9. Form3Difference
10. Form5Home
11. Form5Away
12. Form5Difference
13. NormProbHome
14. NormProbDraw
15. NormProbAway
16. FTResult

4. Model

Match outcome prediction is framed as a multiclass classification problem. Each match is assigned one of three labels: Home Win (H), Away Win (A), or Draw (D). Three models of increasing complexity are trained and evaluated — Logistic Regression, Random Forest, and XGBoost — each trained under two feature set configurations. This progressive approach allows the contribution of individual feature groups to be isolated and compared.

4.1. Target Encoding

The target variable `FTResult` is a categorical string column containing the values H, D, and A. Scikit-learn's `LabelEncoder` was used to map these to integer labels prior to training. The encoder sorts classes alphabetically, producing the following mapping:

Label	Encoded Value
A — Away Win	0
D — Draw	1
H — Home Win	2

The encoder was fitted exclusively on the training set and applied to the validation set using the same mapping to prevent any information from the validation season influencing the encoding.

4.2. Feature Sets

Two feature sets were defined to measure the isolated contribution of betting market data to predictive performance.

Feature Set A — No Odds contains only football-derived features: Elo ratings, derived Elo difference, rolling form over three and five matches, and derived form differences, alongside one-hot encoded home and away team identities. This set answers the question of how well football statistics alone can predict outcomes.

Feature Set B — With Odds extends Feature Set A with three normalised implied probability features: `NormProbHome`, `NormProbDraw`, and `NormProbAway`. These encode the pre-match betting market's assessment of each outcome. This set measures how much predictive power the market contributes beyond raw football statistics.

All three models were trained independently under both feature sets, producing six trained models in total.

4.3. Preprocessing Pipeline

A separate preprocessing pipeline was constructed for each model family using scikit-learn's `Pipeline` and `ColumnTransformer` abstractions. This guarantees that all transformations are

fitted on training data only and applied consistently to the validation set, preventing preprocessing leakage.

4.3.1. Categorical Features

The HomeTeam and AwayTeam columns were encoded using OneHotEncoder. The encoder was configured with handle_unknown="ignore", which silently zeroes out the encoded vector for any team name not seen during training. This is necessary because the 2024/25 season included promoted clubs with no representation in the training data.

For Logistic Regression, drop="first" was applied to remove one redundant dummy column per categorical feature, avoiding the dummy variable trap in a linear model. For tree-based models, no column was dropped, as decision trees are invariant to linear dependence between features.

4.3.2. Numeric Features

A SimpleImputer with median strategy was applied to all numeric features. Two matches in the validation set had missing Elo values due to a promoted club lacking a recorded rating snapshot for the season's opening fixtures. The median Elo from the training set was used to fill these values.

For Logistic Regression, imputation was followed by StandardScaler to standardise all numeric features to zero mean and unit variance. Logistic Regression is sensitive to feature scale, and without standardisation, features with larger ranges would disproportionately influence the decision boundary. Tree-based models are scale-invariant and received imputed features without standardisation.

4.4. Logistic Regression

Logistic Regression is a linear probabilistic classifier. For multiclass problems, the multinomial (softmax) formulation is used, which models the probability of each class jointly rather than through a series of binary decisions.

Given a feature vector $\mathbf{x}$ and a weight matrix $\mathbf{W}$, the probability assigned to class k is:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^\top \mathbf{x}}}$$

where $K = 3$ is the number of classes and $\mathbf{w}_k$ is the weight vector for class k . The model is trained by minimising the cross-entropy loss with L2 regularisation:

$$\mathcal{L} = - \sum_{i=1}^N \log P(y_i \mid \mathbf{x}_i) + \frac{\lambda}{2} \|\mathbf{W}\|_F^2$$

The regularisation strength is controlled by the parameter $C = \frac{1}{\lambda}$. A value of $C = 1.0$ was used, applying moderate regularisation to reduce overfitting without heavily constraining the model.

Logistic Regression serves as the interpretable baseline in this study. Despite its simplicity, it is known to perform competitively on well-engineered tabular features and provides a lower-bound reference against which ensemble methods are compared.

Hyperparameter	Value
C (inverse regularisation strength)	1.0
max_iter	1000
random_state	42

4.5. Random Forest

Random Forest is an ensemble learning method that constructs a large number of decision trees during training and aggregates their predictions. Each tree is trained on a bootstrap sample of the training data (sampling with replacement), and at each split node, only a random subset of features is considered. This combination of bagging and random feature selection decorrelates the individual trees, reducing variance and improving generalisation over a single decision tree.

For a classification task, the final prediction is determined by majority vote across all trees:

$$\hat{y} = \text{mode} \{h_1(x), h_2(x), \dots, h_T(x)\}$$

where T is the number of trees and h_t is the prediction of the t -th tree. Probability estimates are obtained by averaging the per-class vote proportions across all trees.

Random Forest handles non-linear feature interactions naturally without requiring feature scaling, making it well suited to the mixed numerical and categorical structure of the feature set. The `min_samples_leaf=5` constraint requires each leaf node to contain at least five training samples, providing a form of regularisation that prevents individual trees from memorising noise.

Hyperparameter	Value
n_estimators	300
min_samples_leaf	5
n_jobs	-1 (all available cores)
random_state	42

4.6. XGBoost

XGBoost (Extreme Gradient Boosting) is a gradient boosted decision tree framework. Unlike Random Forest, which builds trees independently in parallel, gradient boosting constructs trees sequentially. Each new tree is trained to correct the residual errors left by the ensemble of all

previous trees. This makes boosting particularly effective at capturing complex non-linear patterns in structured tabular data.

The objective function optimised at each boosting round is:

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

where f_t is the tree added at round t , l is the per-sample loss, and Ω is a regularisation term penalising tree complexity:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$$

Here T is the number of leaf nodes and $\mathbf{w}$ are the leaf weights. This explicit regularisation distinguishes XGBoost from classical gradient boosting and helps control overfitting.

For the three-class prediction task, the `multi:softprob` objective was used, which produces calibrated probability estimates for all three outcome classes simultaneously. The learning rate was set conservatively at 0.05 to allow more boosting rounds without overfitting, and subsampling was applied at both the row (`subsample=0.8`) and feature (`colsample_bytree=0.8`) levels to further regularise the model.

Hyperparameter	Value
n_estimators	300
max_depth	4
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
objective	multi:softprob
num_class	3
random_state	42

XGBoost is the most expressive of the three models and is expected to achieve the strongest performance on tabular data. It handles sparse one-hot encoded inputs efficiently and does not require feature scaling, making it a natural fit for the combined categorical and numerical feature structure of this dataset.